Introduction

This `template_methods_paper` serves as the **methods-paper exemplar** for the Research Project Template ecosystem: a manuscript whose subject is a methodology — here, a domain language for specifying controlled methods — rather than results produced by running one. The prose, the labelled figures, and the compiled-plan table are produced through the same auditable custody chain every exemplar in this template uses: tested functions in `src/`, a thin analysis script, and generated-variable-injected, multi-format rendering.

Why a methods paper needs its own DSL I

A methods section in ordinary prose is ambiguous by construction: "add 10 mL of water, then mix" admits multiple readings of order, units, and what "mix" means operationally. BPL (Bota Biosciences 2026) makes the case for laboratory protocols directly: free-text instructions admit multiple interpretations, unit errors and reagent mismatches surface only at the bench, and re-executing a protocol on a different operator or instrument introduces silent variation. Fowler frames the general remedy as a **domain-specific language** (Fowler 2010): a small, purpose-built notation whose vocabulary is restricted exactly to the concepts the domain needs, so that what can be written down is exactly what is intended.

What this exemplar borrows from BPL, and what it generalizes I

BPL's architecture is a compiler pipeline — parse, semantic check, lower, schedule, execute, export — over a biology-native type system (units, dimensional analysis, MW-aware concentration), staged validation gates, and deterministic compilation with a stable plan hash. Three design choices carry over directly into `src/template_methods_paper/methods_dsl/`:

What this exemplar borrows from BPL, and what it generalizes I

1. **Intent over instruction.** BPL users write high-level intents (`transfer`, `add_reagent`, `incubate`); a compiler lowers them to backend-specific primitives. `src/template_methods_paper/methods_dsl/vocabulary.py`'s `StepKind` enum is the same idea, generalized: `TRANSFER`/`ADD`/`MIX` name *what* happens, never *how* a particular backend performs it.
2. **Dimensional safety.** BPL's type system catches `mL + g` at compile time, not at the bench. `src/template_methods_paper/methods_dsl/units.py` implements the same guarantee with a small `Dimension`/`Quantity` system rather than a full unit library.
3. **Deterministic compilation.** Same source, same options, same plan hash. `src/template_methods_paper/methods_dsl/compiler.py::compile_method` reproduces this with a canonical-JSON SHA-256 hash over a Kahn's-algorithm (Kahn 1962) schedule.

What this exemplar borrows from BPL, and what it generalizes I

What this exemplar does **not** carry over is BPL's text grammar and parser: a `Method` here is constructed directly as frozen Python dataclasses (`src/template_methods_paper/methods_dsl/model.py`), not parsed from `.bpl` source. This keeps the DSL's discipline in its typed, validated shape rather than in new concrete syntax — appropriate for a template exemplar's scope — while the controlled vocabulary, dimensional safety, and deterministic compilation generalize unchanged from wet-lab protocols to any controlled procedure, demonstrated in sec. 4 by one wet-lab-flavored method and one instrument-calibration method.

Template architecture context I

The project sits on the repository's three pillars:

Template architecture context I

1. `src/template_methods_paper/methods_dsl/` **library**: pure, side-effect-free dataclasses and functions — no plotting, no file I/O, and (with one declared logging exception) no `infrastructure` imports. This purity is what makes the library forkable and trivially testable.
2. `tests/` **framework**: a zero-mock suite that exercises every gate, the compiler, and the exporters against real `Method` fixtures covering both the success path and every gate-failure mode.
3. `docs/` **knowledge base**: the correspondence with BPL's pipeline, the testing philosophy, and the operational rules that govern agents editing this tree.

The worked examples I

We specify two methods with `all_example_methods()` (`src/template_methods_paper/methods_dsl/examples_methods.py`): `PBSPreparation`, an original — not copied from BPL's shipped examples — manual bench preparation in BPL's own domain, and `SensorCalibrationSweep`, a non-biology controlled procedure mixing automated measurement with a human sign-off step. The second example exists specifically to demonstrate that the DSL's vocabulary generalizes beyond wet-lab protocols, as sec. `sec:introduction` claims.

Reader's guide to the manuscript I

Reader's guide to the manuscript I

- **sec. 3** ties each pipeline stage to its module in `src/template_methods_paper/methods_dsl/`.
- **sec. 4** is artifact-centric: every reported number names the function or report file that produced it.
- **sec. 6** lists the controlled vocabulary and software environment.
- **sec. 7** records the artifact inventory and the exact commands to regenerate everything.
- **sec. 8** states scope and related work so the exemplar is not mistaken for a general-purpose protocol-execution system.